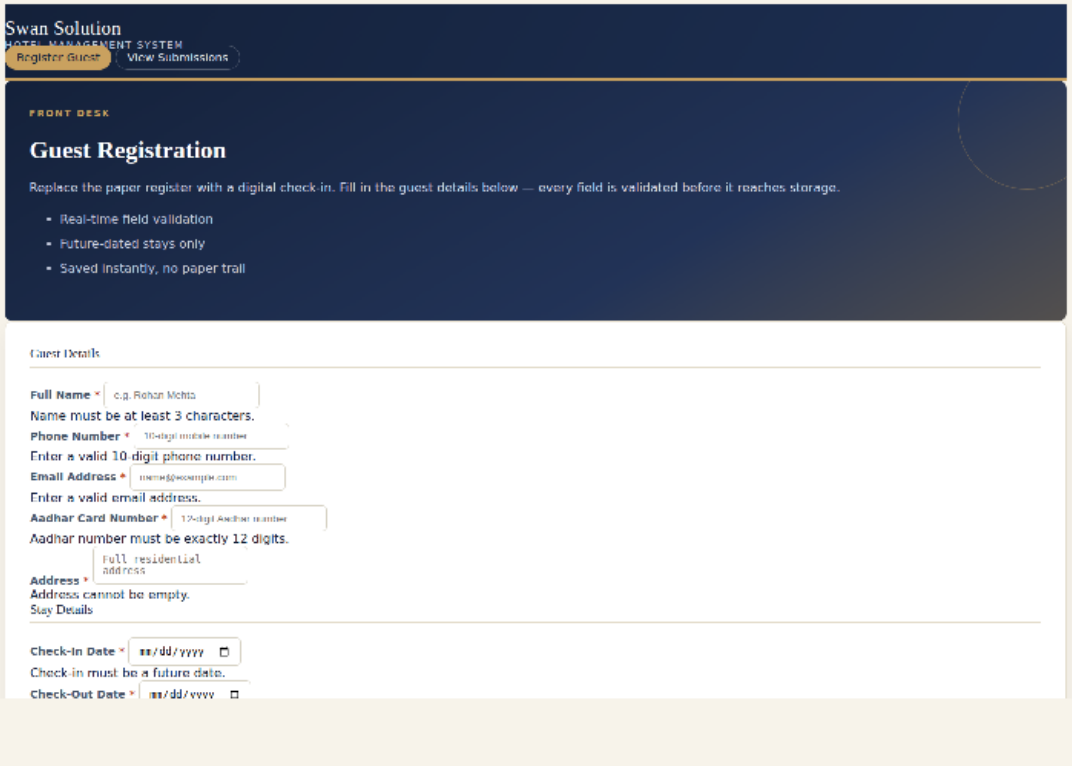
PROJECT SYNOPSIS

on

SWAN SOLUTION

Hotel Management System —Digital Guest Registration Module



Target Organization	Swan Solution Pvt. Ltd. (Hospitality / Hotel Front-Desk Operations)
Document Type	Project Synopsis
Development Paradigm	Object-Oriented, Client-Side Web Application (Incremental /

	Prototyping Model)
Submitted As	Module 8 — JavaScript Essentials and Advanced
Date	2026

Agenda

- Introduction
- Purpose of the Project
- Scope of the Project
- Technologies Used
- Objectives
- System Requirements
- Constraints
- Hardware & Software Components
- Problem Solving Approach
- Advantages & Disadvantages
- Data Dictionary
- System Characteristics
- UML & Structured Design Diagrams
- Output Screenshots
- Conclusion

1. Introduction

Swan Solution is a client-side Hotel Management System built to digitize the front-desk guest registration process that is traditionally maintained on paper registers. The system provides a single-page-style web application consisting of two coordinated screens: a Guest Registration form and a View Submissions dashboard, both backed by the browser's persistent local storage.

The application replaces manual bookkeeping with a validated digital workflow — every guest detail is checked in real time before being committed to storage, reducing data-entry errors, illegible handwriting issues, and the risk of lost paper records. Since the system runs entirely in the browser with no backend server, it is lightweight, fast to deploy, and suitable as a learning project demonstrating JavaScript, DOM manipulation, form validation, and object-oriented front-end architecture.

2. Purpose of Project

The purpose of this project is to provide hotel front-desk staff with a fast, reliable, and paperless way to record guest check-in details and to retrieve, search, and manage those records afterward. It demonstrates how core JavaScript concepts — classes, event delegation, form validation, and browser storage — can be combined to build a small but complete business application without any server-side infrastructure.

3. Scope

The current scope of Swan Solution covers:

- Digital registration of guest personal and stay details through a validated web form.
- Client-side persistence of guest records using the browser's localStorage.
- A searchable, tabular view of all recorded submissions with delete capability.
- Real-time field-level validation (name length, phone/Aadhar digit patterns, email format, future-dated check-in, and check-out-after-check-in logic).

The scope does not currently include: multi-user login/authentication, room allocation and billing, server-side/database storage, or multi-device data synchronization — these are noted as future enhancements.

4. Technologies Used

Layer	Technology	Purpose
Markup	HTML5	Semantic page structure for the registration form and submissions table
Styling	CSS3 (custom) + Bootstrap 5.3.3	Responsive layout, form styling, navy & gold hotel-brand theme

Icons / Fonts	Bootstrap Icons, Google Fonts (Inter, Playfair Display)	Visual polish and iconography
Scripting	Vanilla JavaScript (ES6 Classes)	Form validation, event handling, storage logic, table rendering
Persistence	Browser localStorage API	Client-side data storage in JSON format
Architecture	Object-Oriented JS (CustomerFormHandler, SubmissionViewer classes)	Encapsulation of registration and viewing logic in one shared script.js

5. Objectives

- To eliminate paper-based guest registration and replace it with a digital, validated form.
- To enforce data integrity through real-time client-side validation before any record is saved.
- To allow front-desk staff to quickly search and review previously registered guests.
- To provide the ability to remove incorrect or duplicate entries.
- To build the application using clean, reusable, object-oriented JavaScript that can later be extended with a real backend and database.

6. System Requirements

6.1 Functional Requirements

- The system shall allow entry of guest details: full name, phone, email, Aadhar number, and address.
- The system shall allow entry of stay details: check-in date, check-out date, number of adults, and purpose of visit.
- The system shall validate every field before allowing form submission.
- The system shall persist a successfully validated record without a page reload.
- The system shall display all saved records in a searchable table.
- The system shall allow deletion of an individual record from the table.

6.2 Non-Functional Requirements

- Usability: form should provide instant visual feedback (valid/invalid) as the user types.
- Performance: rendering and search filtering should feel instantaneous for typical front-desk record volumes.
- Portability: the system should run in any modern browser without installation.
- Maintainability: logic should be organized into classes with clear single responsibilities.

7. Constraints

- Data is stored only in the browser's localStorage, so records are local to a single browser/

device and are not shared across machines.

- Clearing browser data/cache will permanently erase all saved guest records.
- There is no authentication layer — anyone with access to the browser can view or delete records.
- The system does not currently validate for duplicate guests or overlapping room bookings.
- localStorage has a practical size limit (typically 5–10 MB per origin), which caps the number of records over the long term.

8. Hardware & Software Components

8.1 Hardware Requirements

Component	Minimum Specification
Processor	1 GHz or faster (any modern PC/laptop/tablet)
RAM	2 GB or higher
Storage	Minimal — a few MB for browser cache
Display	Any screen supporting a modern web browser (desktop or mobile)

8.2 Software Requirements

Component	Requirement
Operating System	Windows / macOS / Linux / Android / iOS (any OS with a modern browser)
Browser	Google Chrome, Microsoft Edge, Firefox, or Safari (latest versions)
Frameworks / Libraries (CDN)	Bootstrap 5.3.3, Bootstrap Icons 1.11.3, Google Fonts
Development Tools	Any code/text editor (e.g., VS Code); no server or database installation required

9. Problem Solving Approach

The manual, paper-based guest register is slow, error-prone, and difficult to search. Swan Solution addresses this through the following approach:

- Decompose the workflow into two independent, single-responsibility pages: registration and viewing.
- Encapsulate registration logic inside a CustomerFormHandler class that owns field validators, event binding, and submission handling.
- Encapsulate table logic inside a SubmissionViewer class that owns loading, filtering, rendering, and deletion of records.
- Share a common set of storage helper functions (getItems, setItems, addItem, removeItem) so both classes read/write the same data source consistently.

- Use event delegation (single listeners on the form and table body) instead of many individual listeners, keeping the code efficient and easy to maintain.
- Validate every field both on input/blur (immediate feedback) and again on submit (final guard) before anything is persisted.

10. Advantages & Disadvantages

10.1 Advantages

- No installation or server setup — runs directly in a browser.
- Instant, real-time validation reduces bad or incomplete data.
- Simple, clean, object-oriented codebase that is easy to extend.
- Fast search/filter over existing records without any network round-trip.
- Responsive Bootstrap-based UI works on desktop and mobile devices.

10.2 Disadvantages

- Data is confined to a single browser/device — no centralized or multi-user access.
- No authentication, so records are not protected from unauthorized viewing or deletion.
- No backend database — data can be lost if browser storage is cleared.
- No billing, room-allocation, or reporting modules in the current scope.

11. Data Dictionary

The system stores each guest submission as a single JSON object under the localStorage key "swan_hotel_guests". The structure of one guest record is described below:

Field Name	Data Type	Constraint	Description
id	String	Auto-generated, Primary Key	Unique identifier created from timestamp + random string
createdAt	ISO DateTime	Auto-generated	Timestamp when the record was saved
fullName	String	Required, min 3 chars	Guest's full name
phone	Numeric String	Required, exactly 10 digits	Guest's contact number
email	String	Required, valid email pattern	Guest's email address
aadhar	Numeric String	Required, exactly 12 digits	Government ID (Aadhar) number
address	String	Required, non-empty	Guest's residential address
checkIn	Date	Required, must be a future date	Planned check-in date
checkOut	Date	Required, after checkIn	Planned check-out date

adults	Number	Required, >= 1	Number of adults staying
purpose	String	Required, non-empty	Purpose of visit

12. System Characteristics

- Client-side only: no server, API, or database dependency.
- Object-oriented: core behaviour is encapsulated in ES6 classes (CustomerFormHandler, SubmissionViewer).
- Reactive validation: uses input/blur/change event listeners for instant feedback.
- Persistent: data survives page refresh via localStorage (until browser data is cleared).
- Responsive: Bootstrap grid adapts the layout to desktop, tablet, and mobile screens.
- Searchable: live filtering by guest name or check-in/check-out date.
- Extensible: shared script.js is written so a future version could swap localStorage for a REST API with minimal changes to the class interfaces.

13. Use Case Diagram

The Receptionist (front-desk user) is the sole actor interacting with the system. The primary use cases are registering a guest (which includes validating the form and saving the record), viewing submissions, searching records, and deleting a record.

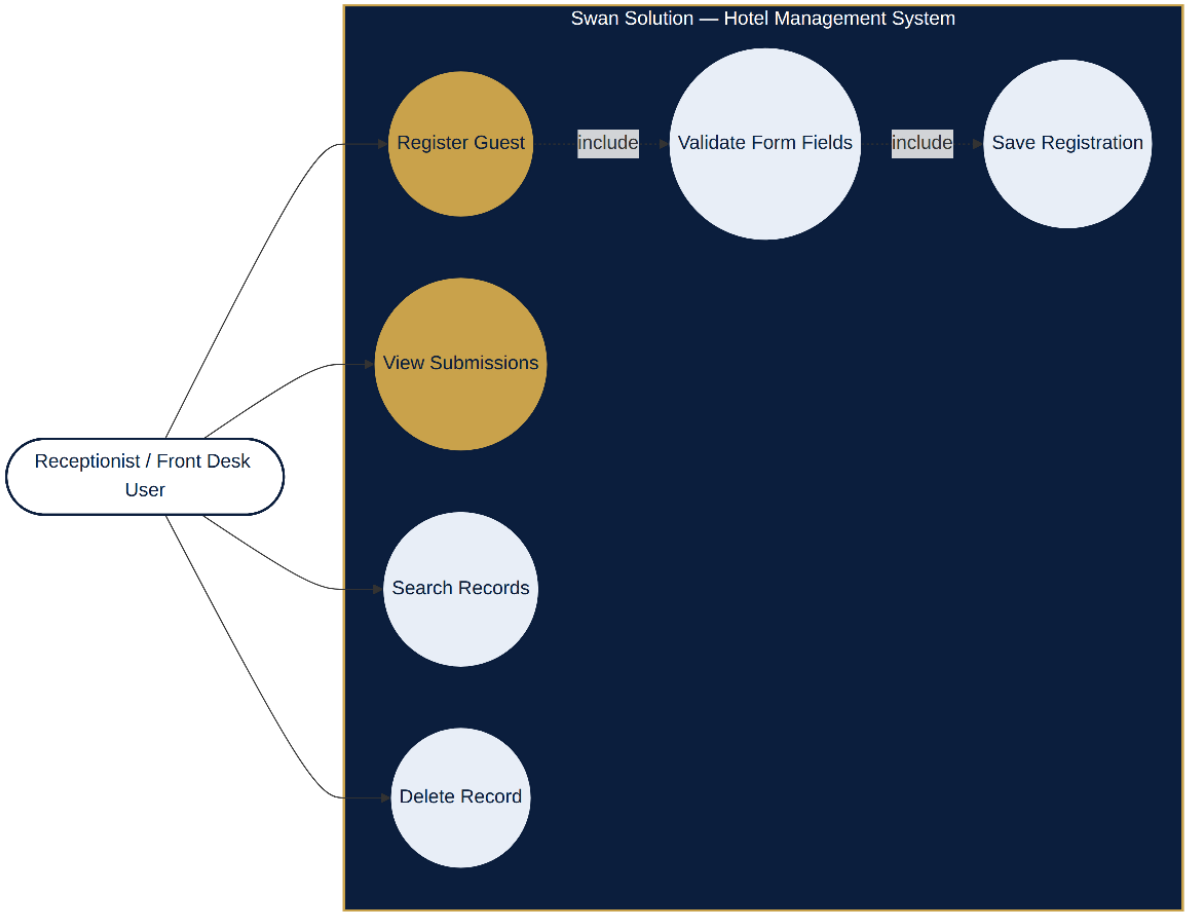


Fig 13: Use Case Diagram

14. Activity Diagram

The activity diagram traces the end-to-end flow from opening the registration page, through field validation (looping back on error), saving the record, and on to viewing, searching, and optionally deleting records.

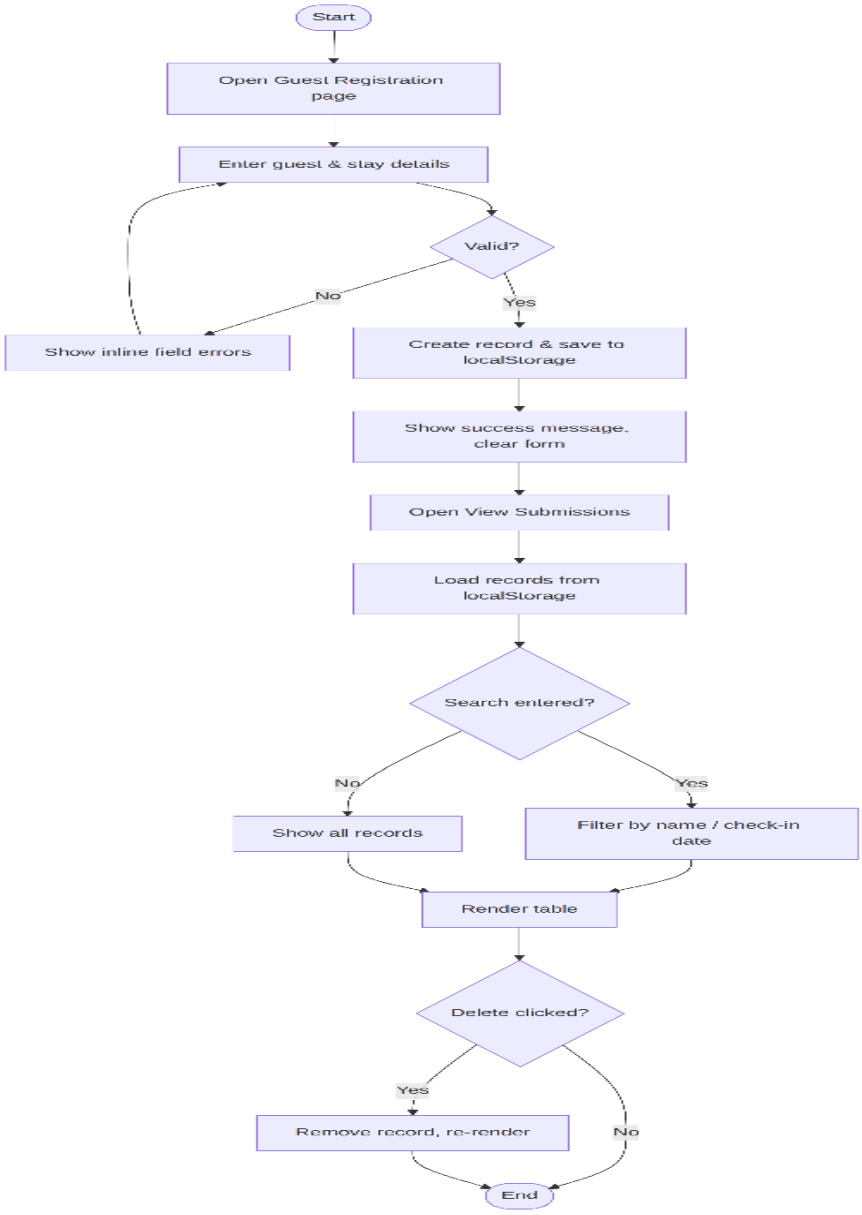


Fig 14: Activity Diagram

15. Sequence Diagram

The sequence diagram shows the message flow between the Receptionist, the Guest Form, the CustomerFormHandler, localStorage, and the SubmissionViewer across both the registration and viewing/search/delete interactions.

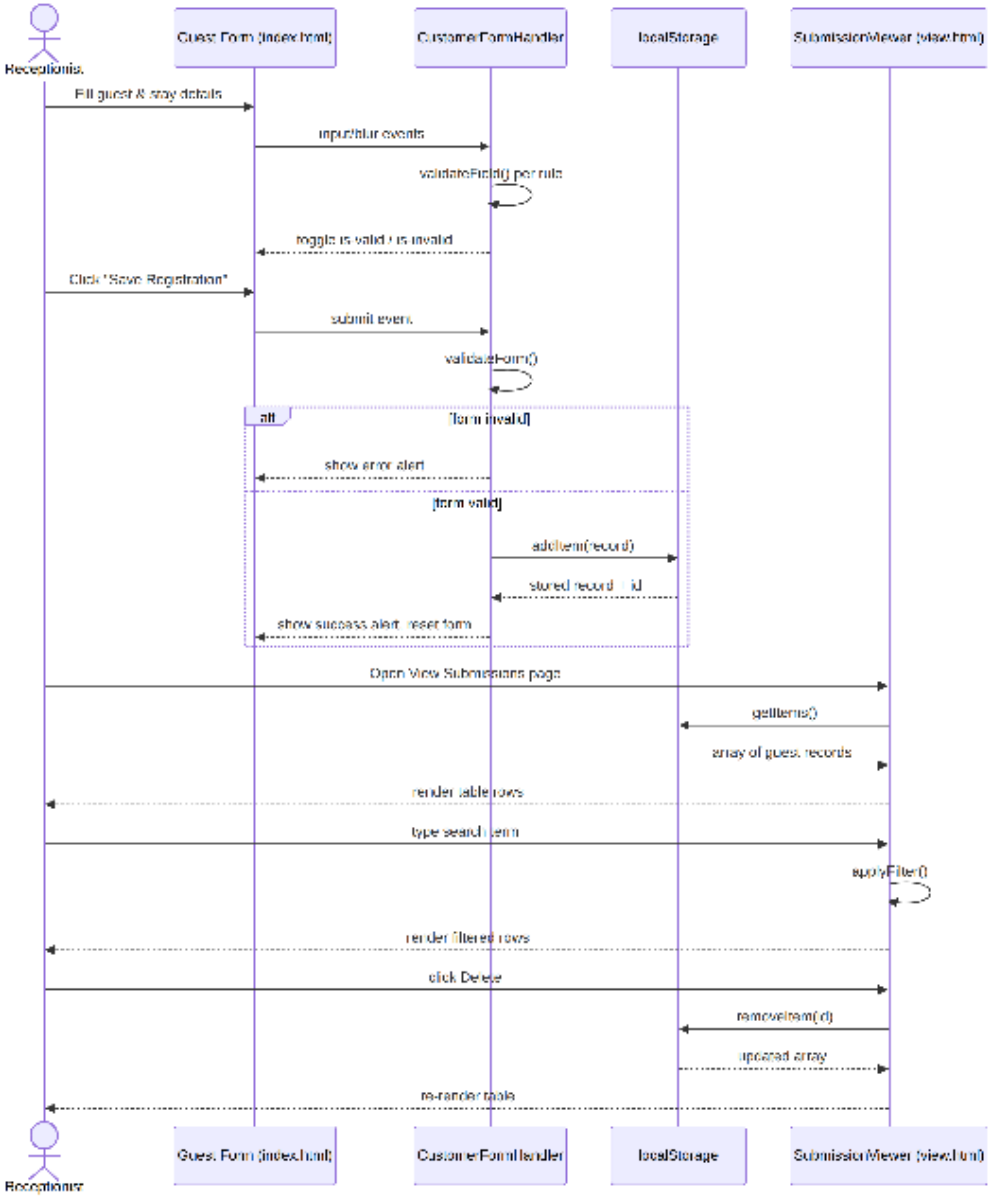


Fig 15: Sequence Diagram

16. ER Diagram

Since the system uses a single flat data store, the ER diagram models one primary entity, GUEST_REGISTRATION, produced by the FRONT_DESK actor's browser session and persisted as a JSON array in localStorage.

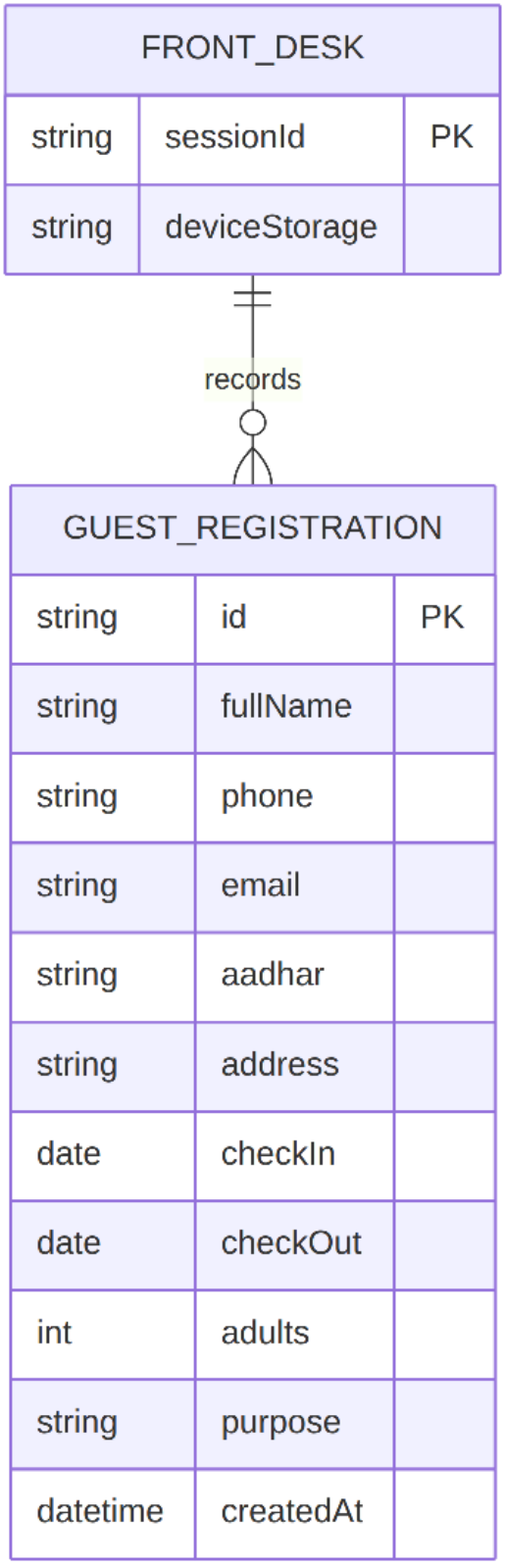


Fig 16: ER Diagram

17. Class Diagram

The class diagram shows the object-oriented structure of script.js: shared StorageHelpers functions, the CustomerFormHandler and SubmissionViewer classes, and the GuestRecord data shape they operate on.

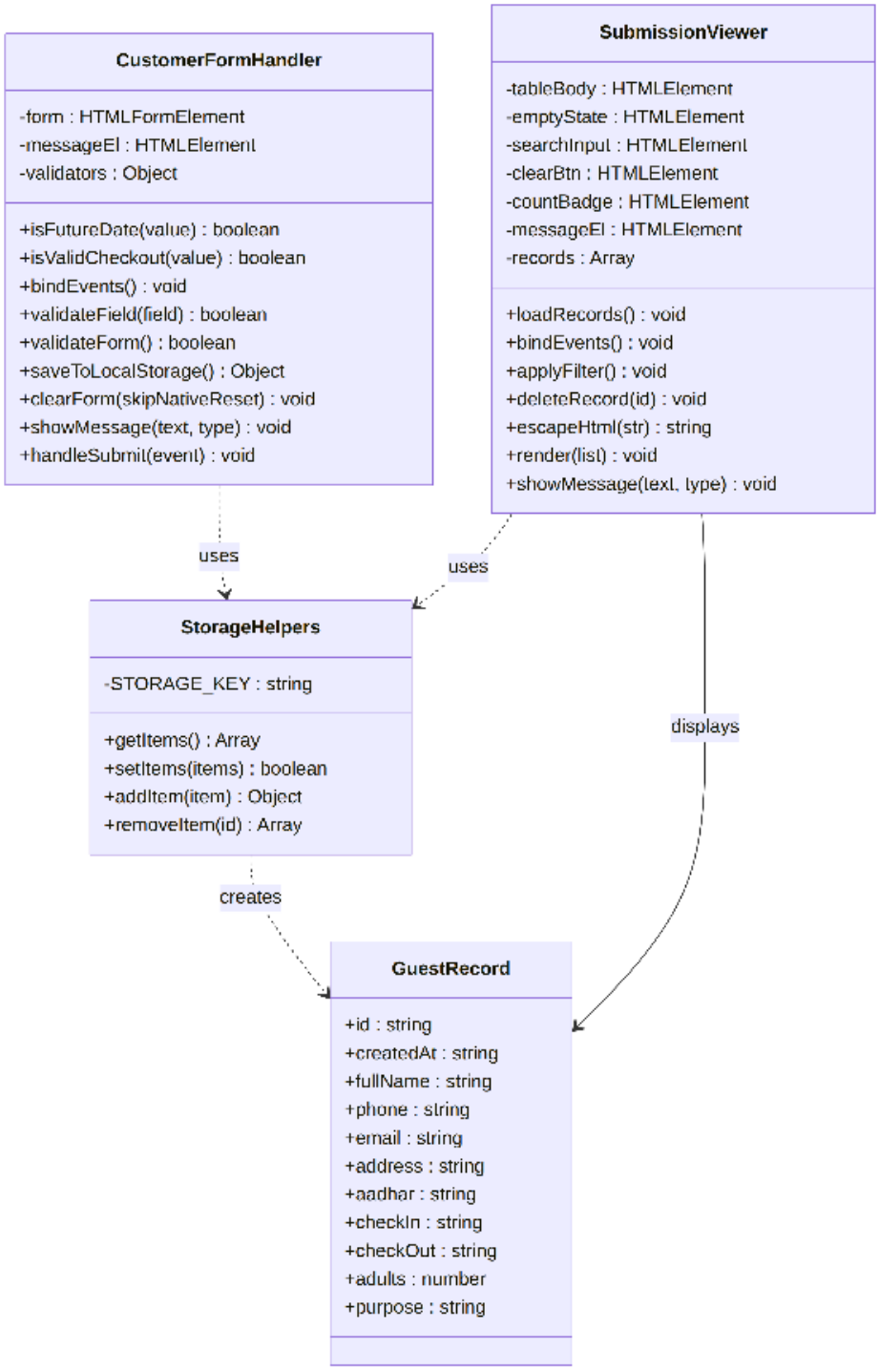


Fig 17: Class Diagram

18. Data Flow Diagram (DFD Level 1)

The DFD illustrates how guest data flows from the Receptionist through validation and storage processes into the localStorage data store, and back out again through the search and delete processes.

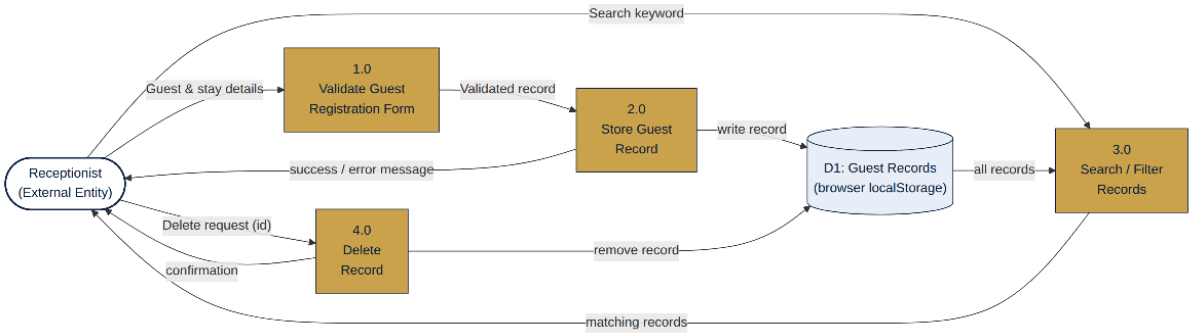


Fig 18: Data Flow Diagram (DFD Level 1)

19. Output Screenshots

Guest Registration screen — the front-desk user fills in guest and stay details; every field is validated live as shown by the green/red field borders.

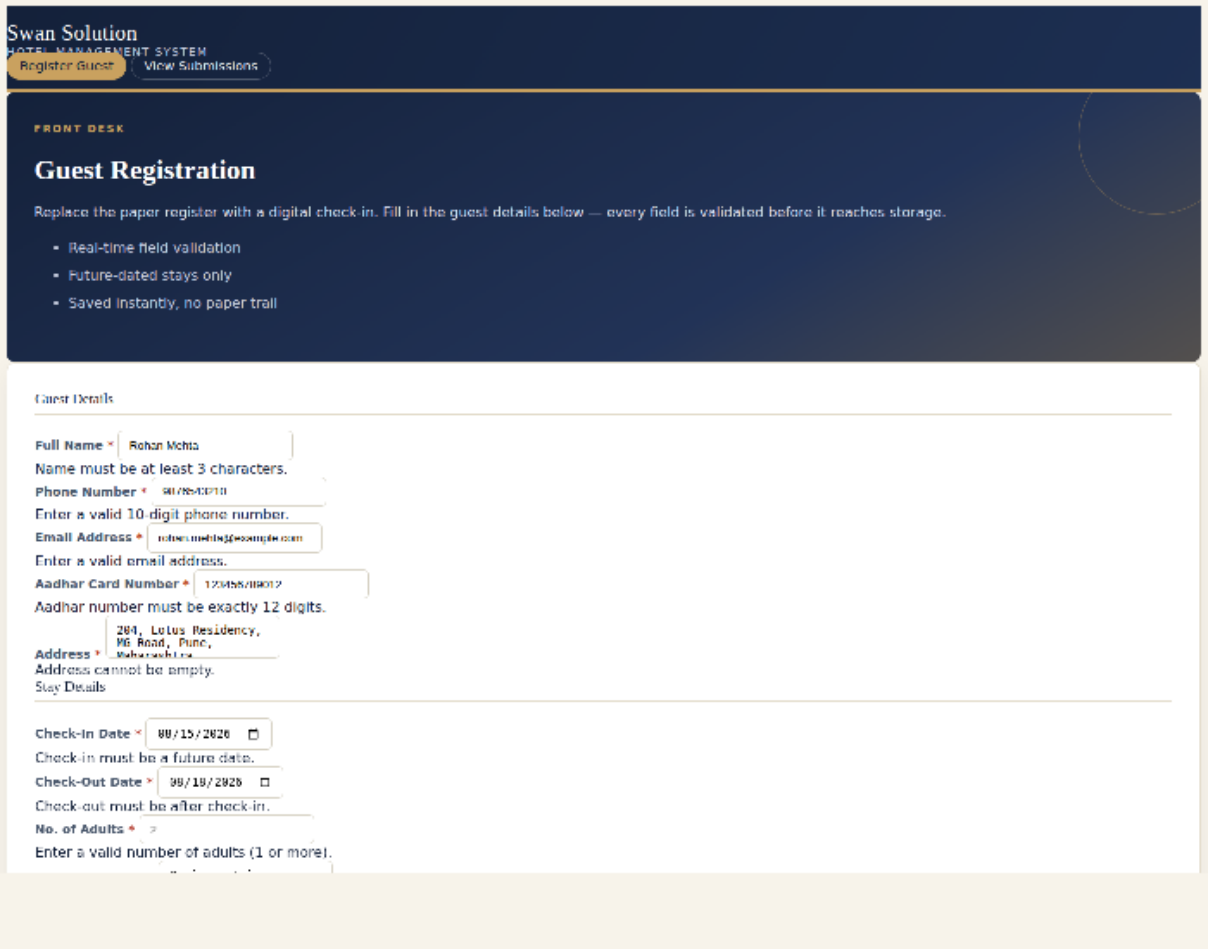


Fig 19a: Guest Registration form with valid data entered

View Submissions screen — all saved guest records are listed in a searchable, sortable-by-recency table, with a delete action available for each row.

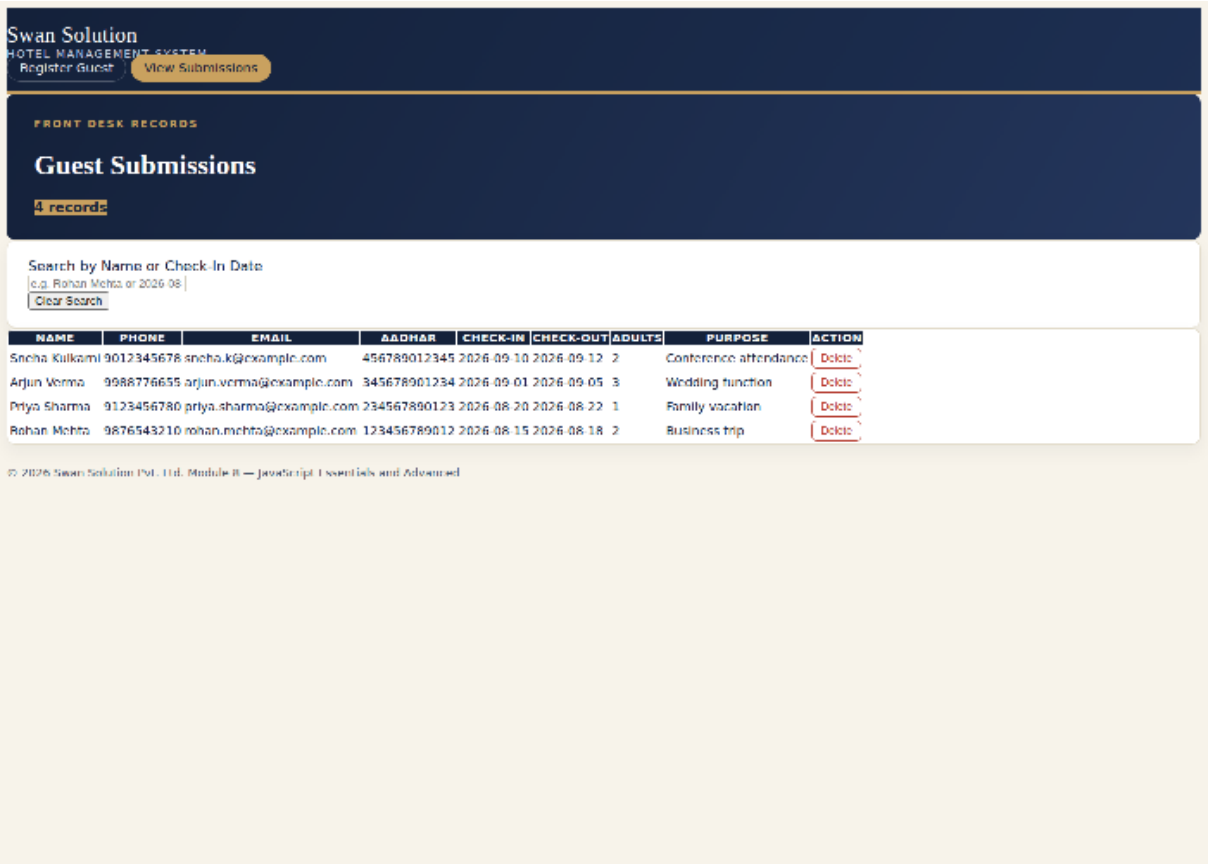


Fig 19b: View Submissions table with sample guest records

20. Conclusion

Swan Solution successfully demonstrates how a purely client-side, object-oriented JavaScript application can digitize a real front-desk workflow — guest registration — with real-time validation, persistent local storage, and a clean searchable records view. The project fulfils its stated objectives within its defined scope, and its modular class-based design (CustomerFormHandler, SubmissionViewer, and shared storage helpers) provides a solid foundation for future enhancements such as backend integration, authentication, room and billing management, and multi-device synchronization.